



Language Models & Evaluation Metrics

IFN647 - Week 5

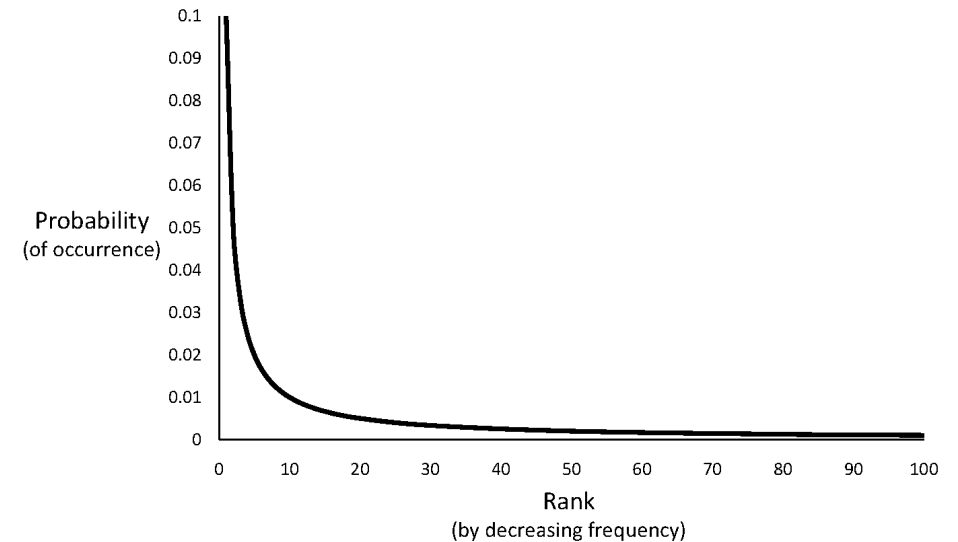
Professor Yuefeng Li
School of Computer Science
Queensland University of Technology

Learning Objectives

Topic ID	Key topics	Mapping to slides	Activities
1	Introduction of Language Models	2-4	Q2
2	Query-Likelihood Model	5-8	Q2
3	Smoothing Technique	9-14	Q1
4	Evaluation Overview and Relevance Judgments	15-22	Q4
5	Effectiveness Measures	23-25	Q4
6	Efficiency Metrics	26-27	
7	Significance Tests	28-32	
	References	33	

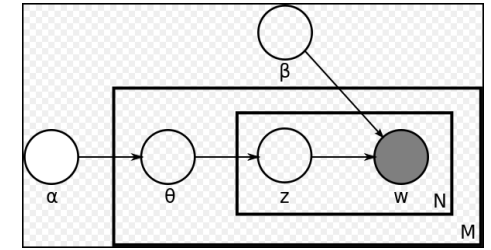
1. Introduction of Language Models

- Language models are used to represent text in a variety of language technologies, such as speech recognition, machine translation, and handwriting recognition. They are the foundation of Large Language Model (LLM).
- The simplest form of language model, known as *Unigram Language Model*
 - It is a probability distribution over the words in a language, e.g., the Zipf's word distribution mentioned in week 2.
 - For a simple example, if the documents in a collection contained just five different words ($w_1, w_2, \dots, w_5$), a possible language model for that collection might be
 - (0.2, 0.1, 0.35, 0.25, 0.1)
 - where each number is the probability of a word occurring.
- N-gram language model
 - predicts words based on longer sequences are used. An n -gram model predicts a word based on the previous $n-1$ **words**.
 - The most common n -gram models are bigram (predicting based on the previous *word*) and trigram (predicting based on the previous two words) models.



Topics and Language Models

3



- We can use language models to represent the topical content of a document.
- A topic is defined as a probability distribution over words – a language model.
- A language model can be used to “**generate**” new text by sampling words according to the probability distribution.
- A *topic* in a document or query can be represented as a language model
 - i.e., words that tend to occur often when discussing a topic will have high probabilities in the corresponding language model.
- The Latent Dirichlet Allocation (LDA) model is a popular model for topic modeling which comes from the machine learning community.
 - LDA assumes that there is a fixed number of underlying (or latent) topics that can be used to describe the contents of documents.
 - Each document is represented as a mixture of these topics.
 - It can calculate language model probabilities $P_{lda}(w | D)$ for the words w in documents D (i.e., words Dirichlet distribution in documents with parameter β):

$$P_{lda}(w|D) = P(w|\theta_D, \beta) = \sum_z P(w|z, \beta)P(z|\theta_D)$$

Language Models for IR

- Three approaches for leveraging language models in IR to estimate the probability of a document being relevant to a given query.
 1. The probability of generating the query text from a document language model.
 2. The probability of generating the document text from a query language model.
 3. Comparing the language models representing the query and document topics.
- Topical relevance for using the first approach
 - The probability of query generation is the measure of how likely it is that a document is about the same topic as the query.
 - It is the query likelihood retrieval model.

2. Query-Likelihood Model

- It ranks documents by the probability that the query could be generated by the document model (i.e., same topic)
- Given query Q , we start with $P(D|Q)$
- Using Bayes' Rule
$$p(D|Q) \stackrel{rank}{=} P(Q|D)P(D)$$
- Assuming priori $P(D)$ is uniform, we can have the simple unigram model, which only use single terms' probabilities $P(q_i | D)$:

$$P(Q|D) = \prod_{i=1}^n P(q_i|D)$$

Estimating Probabilities

- Obvious estimate for unigram probabilities is

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

- where $f_{q_i,D}$ is the number of times query word q_i occurs in document D .
- If query words are missing from document, score will be zero
 - This is the major problem.
 - E.g., missing 1 out of 4 query words same as missing 3 out of 4.
 - This is clearly not appropriate for longer queries.

Python solution

- Language models can be very efficient since it can be implemented over inverted index of documents.
- Firstly, we need to define a function *index_docs(inputpath, stop_words)*
 - which takes a set of documents in *inputpath* folder.
 - It also returns an index – a Python dictionary $I = \{\text{term1}:\{\text{docID1}:\text{freq1}, \text{DocID2}:\text{freq2}, \dots\}, \text{term2}:\{\text{docID1}:\text{freq1}, \text{DocID2}:\text{freq2}, \dots\}, \dots\}$,
- We then define the language model, the function *likelihood_IR(I, Q)* to estimate $P(Q|D)$ for all documents D in the folder indexed by I for the given query Q .

```
def likelihood_IR(I, Q): #index I is a dictionary of term:(itemId:freq)
    L={} # L is the selected inverted list
    R={} # R is a dictionary of docId:score
    D_len={} # D_len is a dictionary of docId:length
    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1 # get all document IDs and initialize as 1
            D_len[id[0]]=0.5 # a small non-zero value as a denominator
        if (list[0] in Q): # select inverted lists based on the query Q
            L[list[0]]= I[list[0]]
    for q_term in Q.items(): # L may not contain all query terms
        if not(q_term[0] in L):
            L[q_term[0]]={}
    for list in I.items():
        for id in list[1].items(): #Count term occurrences in documents
            D_len[id[0]]= D_len[id[0]] + id[1]
    for (d, sd) in R.items():
        for (term, f) in L.items():
            if not(d in f):
                f[d]=0
            sd = sd*(f[d]/D_len[d]) #see the equation in the lecture notes
    R[d] = sd
    return R
```

Test the functions

```
curr_path=os.getcwd()
stopwords_f = open('common-english-words.txt', 'r')
stop_words = stopwords_f.read().split(',')
stopwords_f.close()
Test_path = curr_path+'/Test_docs'
Index_Test = index_docs(Test_path, stop_words) #Index all terms
os.chdir(curr_path)
Query = {'share':1, 'bank':1}
#test the likelihood IR model
IR_result = likelihood_IR(Index_Test, Query)
y = sorted(IR_result.items(), key=lambda x: x[1],reverse=True)
print('Likelihood Query Result -----')
for (id, w) in y:
    if w>0:
        print('Document ID: '+id + ' and relevance weight: ' + str(w))
```

- Outputs

Likelihood Query Result -----

Document ID: 807606 and relevance weight: 0.0009652315550282532

Document ID: 807600 and relevance weight: 0.00014154245894636803

3. Smoothing Technique

- Document texts are a *sample* from the language model
 - Missing words should not have zero probability of occurring
- **Smoothing** is a technique for estimating probabilities for missing (or unseen) words [1]
 - lower (or *discount*) the probability estimates for words that are seen in the document text
 - assign that “left-over” probability to the estimates for the words that are not seen in the text

Smoothing for Query-Likelihood

- Estimate probabilities for unseen words in a document D is $\alpha_D P(q_i|C)$, where
 - $P(q_i|C)$ is the probability for query word i in the *collection* language model for collection C (background probability)
 - α_D is a parameter
- Then, the estimate for words that occur in D is
$$(1 - \alpha_D) P(q_i|D) + \alpha_D P(q_i|C)$$
- Different forms of estimation come from different α_D

Jelinek-Mercer (JM) Smoothing

- α_D is a constant, λ
- Gives the equation for estimating query term's probability:

$$p(q_i|D) = (1 - \lambda) \frac{f_{q_i,D}}{|D|} + \lambda \frac{c_{q_i}}{|C|}$$

- where c_{q_i} is the number of times a query word occurs in C , and $|C|$ is the total number of word occurrences in collection C .
- When λ is small, less smoothing is applied because the second component becomes small.
- The equation for the ranking score

$$P(Q|D) = \prod_{i=1}^n ((1 - \lambda) \frac{f_{q_i,D}}{|D|} + \lambda \frac{c_{q_i}}{|C|})$$

- Use logs for convenience
 - accuracy problems for multiplying small numbers

$$\log P(Q|D) = \sum_{i=1}^n \log((1 - \lambda) \frac{f_{q_i,D}}{|D|} + \lambda \frac{c_{q_i}}{|C|})$$

An Example

- Let $Q = \{\text{US}, \text{ECONOM}, \text{ESPIONAG}\}$ be a query, and $C = \{D_1, D_2, D_3, D_4, D_5, D_6\}$ be a collection of documents, where
 - $D_1 = \{\text{GERMAN}, \text{VW}\}$
 - $D_2 = \{\text{US}, \text{US}, \text{ECONOM}, \text{SPY}\}$
 - $D_3 = \{\text{US}, \text{BILL}, \text{ECONOM}, \text{ESPIONAG}\}$
 - $D_4 = \{\text{US}, \text{ECONOM}, \text{ESPIONAG}, \text{BILL}\}$
 - $D_5 = \{\text{GERMAN}, \text{MAN}, \text{VW}, \text{ESPIONAG}, \text{VW}\}$
 - $D_6 = \{\text{GERMAN}, \text{GERMAN}, \text{MAN}, \text{VW}, \text{SPY}, \text{GERMAN}\}$
- Let $D = \{\text{US}, \text{ECONOM}, \text{SPY}\}$ be an incoming document, calculate $P(Q|D)$.

An Example cont.

- We firstly need to estimate each query term's probability $p(Q_i | D)$
 - $p(\text{US} | D) = (1-0.5) * (1/3) + 0.5 * (4/25) = 0.167 + 0.08 = 0.247$
 - $p(\text{ECONOM} | D) = 0.5 * (1/3) + 0.5 * (3/25) = 0.167 + 0.06 = 0.227$
 - $p(\text{ESPIONAG} | D) = 0.5 * (0/3) + 0.5 * (3/25) = 0 + 0.06 = 0.06$
- *By using the ranking score equation, we have*

$$p(Q | D) = p(\text{US} | D) * p(\text{ECONOM} | D) * p(\text{ESPIONAG} | D) =$$

$$0.247 * 0.227 * 0.06 = 0.00336414$$
- The result is a very small value (the accuracy problem for multiplying small numbers)
- In practical, we could use log function (base = 10), that is,

$$\log(P(Q | D)) = \log(p(\text{US} | D)) + \log(p(\text{ECONOM} | D)) + \log(p(\text{ESPIONAG} | D)) =$$

$$\log(0.247) + \log(0.227) + \log(0.06) = -2.473$$

Implementation of JM smoothing model

- We need two folders:
 - Test_docs # the incoming documents
 - Collection_C # a document collection for conducting smoothing
- *likelihood_IR*(I, Q) can be used by updating the calculation for $P(Q|D)$ and $P(q_i|D)$:
 1. Calculate query term frequency in Collection_C.
 2. Use the formula from the previous page.

4. Evaluation Overview and Relevance Judgments

- Evaluation is key to building *effective* and *efficient* search systems
 - measurement usually carried out in controlled laboratory experiments
 - *online* testing can also be done
- Effectiveness, efficiency and *cost* are related
 - e.g., if we want a particular level of effectiveness and efficiency, this will determine the cost of the system configuration
 - efficiency and cost targets may impact effectiveness

Examples of Evaluation Corpus

- One of the basic requirements for evaluation is that the results from different techniques can be compared.
- To do this comparison fairly and to ensure that experiments are repeatable, the experimental settings and data used must be fixed.
- Test collections for an IR or a ML system consisting of documents, relevance judgments, or queries, e.g., (TREC - The Text Retrieval Conference, <https://trec.nist.gov/>)
- CACM: Titles and abstracts from the Communications of the ACM from 1958-1979. Queries and relevance judgments generated by computer scientists.
- AP: Associated Press newswire documents from 1988-1990 (from TREC disks 1-3). Queries are the title fields from TREC topics 51-150. Topics and relevance judgments generated by government information analysts.
- GOV2: Web pages crawled from websites in the .gov domain during early 2004. Queries are the title fields from TREC topics 701-850. Topics and relevance judgments generated by government analysts.

Collection	Number of documents	Size	Average number of words/doc.
CACM	3,204	2.2 Mb	64
AP	242,918	0.7 Gb	474
GOV2	25,205,179	426 Gb	1073

Collection	Number of queries	Average number of words/query	Average number of relevant docs/query
CACM	64	13.0	16
AP	100	4.3	220
GOV2	150	3.1	180

TREC Topic (Query) Example

<top>

<num> Number: 794

<title> pet therapy

<desc> Description:

How are pets or animals used in therapy for humans and what are the benefits?

<narr> Narrative:

Relevant documents must include details of how pet- or animal-assisted therapy is or has been used. Relevant details include information about pet therapy programs, descriptions of the circumstances in which pet therapy is used, the benefits of this type of therapy, the degree of success of this therapy, and any laws or regulations governing it.

</top>

Relevance Judgments

- The relevance judgments depend on the task that is being evaluated.
 - E.g., TREC judgments
 - generally binary
 - agreement good because of “narrative”
 - For topical relevance, TREC analysts need to judge a document as relevant if it contains information that could be used to help write a report on the query topic.
- Obtaining relevance judgments is an expensive, time-consuming process
 - who does it?
 - what are the instructions?
 - what is the level of agreement?
- There are several possible way to generate (automaticlly) or create new test collections: Pooling, Logging.

Pooling

- Exhaustive judgments for all documents in a collection is not practical.
- Pooling technique is used in TREC
 - top k results (for TREC, k varied between 50 and 200) from the rankings obtained by different search engines (or retrieval algorithms) are merged into a pool,
 - duplicates are removed,
 - documents are presented in some random order to the relevance judges
- Produces a large number of relevance judgments for each query, although still incomplete.

- Query logs that capture user interactions with a search engine (or AI systems) have become an extremely important resource for web search engine (or Open AI projects) development.
- They can be used for both tuning and evaluating search engines
 - also, for various techniques such as query suggestion
- Typical contents
 - User identifier or user session identifier
 - Query terms - stored exactly as user entered
 - List of URLs of results, their ranks on the result list, and whether they were clicked on
 - Timestamp(s) - records the time of user events such as query submission, clicks.
- The **privacy** (e.g., User identifier) is normally a concern when query logs are shared, distributed for research, or used to construct user profiles.
 - Various techniques can be used to anonymize the logged data,
 - removing identifying information or queries that may contain personal data, although this can reduce the utility of the log for some purposes.
 - Federated learning, which trains a machine learning algorithm on multiple local datasets contained in local nodes without explicitly exchanging data samples. The global model can be generated by sharing all nodes or exchanging parameters between local nodes.

Implicit Feedback in Query Logs

- Implicit feedback, such as Clicks, are not relevance judgments
 - although they are correlated
 - biased by a number of factors such as rank on result list
- But we can use clickthrough data to predict *preferences* between pairs of documents
 - appropriate for tasks with multiple levels of relevance, focused on user relevance
 - various “policies” used to generate preferences
- *Skip Above and Skip Next - Policy*
 - click data

d_1
 d_2
 d_3 (clicked)
 d_4

- generated preferences

$$\begin{aligned}
 d_3 &> d_2 \\
 d_3 &> d_1 \\
 d_3 &> d_4
 \end{aligned}$$

Filtering Clicks

- Click data can also be aggregated to remove noise
- *Click distribution* information
 - can be used to identify clicks that have a higher frequency than would be expected
 - high correlation with relevance
 - e.g., using click deviation to filter clicks for preference-generation policies.
- *Click deviation* $CD(d, p)$ for a result d in position p :

$$CD(d, p) = O(d, p) - E(p)$$

where

$O(d, p)$: observed click frequency for a document d in a rank position p **over all instances of a given query.**

$E(p)$: expected click frequency at rank p **averaged across all queries.**

5. Effectiveness Measures

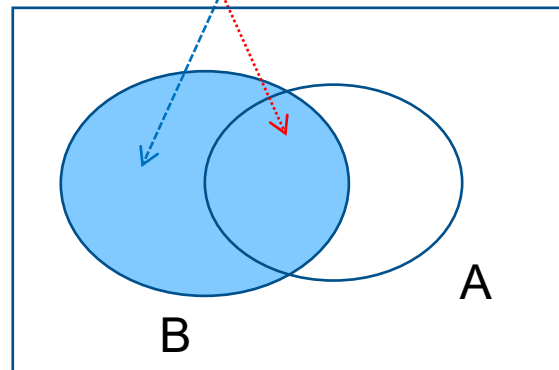
Assume

- A is set of relevant documents, coming from *relevance judgments*, and
- B is set of retrieved documents (the rest of the documents are called *Not Retrieved*), coming from an IR system.

	Relevant	Non-Relevant
Retrieved	$A \cap B$	$\overline{A} \cap B$
Not Retrieved	$A \cap \overline{B}$	$\overline{A} \cap \overline{B}$

$$Recall = \frac{|A \cap B|}{|A|}$$

$$Precision = \frac{|A \cap B|}{|B|}$$



Recall and *precision* are the most common effectiveness measures:

- *recall* measures how well the IR system is doing at finding all the relevant documents for a query.
- *precision* measures how well it is doing at rejecting non-relevant documents.

Classification Errors

- *False Positive* (Type I error)
 - a non-relevant document is retrieved

$$Fallout = \frac{|\bar{A} \cap B|}{|\bar{A}|}$$

- *False Negative* (Type II error)
 - a relevant document is not retrieved
 - 1- *Recall*
- *Precision* is used when probability that a positive result is correct is important.

F Measure

- It is *Harmonic mean* of recall and precision

$$F = \frac{1}{\frac{1}{2}(\frac{1}{R} + \frac{1}{P})} = \frac{2RP}{(R+P)}$$

- harmonic mean emphasizes the importance of small values, whereas the arithmetic mean is affected more by outliers that are unusually large
- More general form
$$F_{\beta} = (\beta^2 + 1)RP/(R + \beta^2 P)$$
- β is a parameter that determines relative importance of recall and precision. Values of $\beta > 1$ emphasize recall.
- The common F measure is in fact F_1 ($\beta = 1$) where recall and precision have equal importance.

6. Efficiency Metrics

- Compared to effectiveness, the efficiency of a search system seems like it should be easier to quantify.
- Most of what we care about can be measured automatically with a timer instead of with costly relevance judgments.
- However, like effectiveness, it is important to determine exactly what aspects of efficiency we want to measure.

Efficiency Metrics cont.

Metric name	Description
Elapsed indexing time	Measures the amount of time necessary to build a document index on a particular system.
Indexing processor time	Measures the CPU seconds used in building a document index. This is similar to elapsed time, but does not count time waiting for I/O or speed gains from parallelism.
Query throughput	Number of queries processed per second.
Query latency	The amount of time a user must wait after issuing a query before receiving a response, measured in milliseconds. This can be measured using the mean, but is often more instructive when used with the median or a percentile bound.
Indexing temporary space	Amount of temporary disk space used while creating an index.
Index size	Amount of storage necessary to store the index files.

CRICOS No.00213J

7. Significance Tests

- Given the results from a number of queries, how can we conclude that ranking algorithm B is better than algorithm A ?
 - A significance test enables us to
 - reject the *null hypothesis* (there is *no difference* in effectiveness between the two retrieval algorithms).
 - in favor of the *alternative hypothesis* (there is *statistical difference* between A and B).
 - If the mean effectiveness of algorithm B is higher than that of algorithm A , and a significance test rejects the null hypothesis of no difference, we can conclude that algorithm B performs statistically better than algorithm A .
-
- A Type I error is when the null hypothesis is rejected when it is in fact true.
 - A Type II error is when the null hypothesis is accepted when it is in fact false.
 - The *power* of a test is the probability that the test will reject the null hypothesis correctly
 - A test with high power will reduce the chance of a Type II error.
 - Increasing the number of queries in the experiment also increases power of test; and also reduce the chance of a Type I error as well.
 - If the probability of the null hypothesis being true is very small, we reject that hypothesis and conclude that
 - ranking algorithm B is more effective than the baseline algorithm A .

CRICOS No.00213J

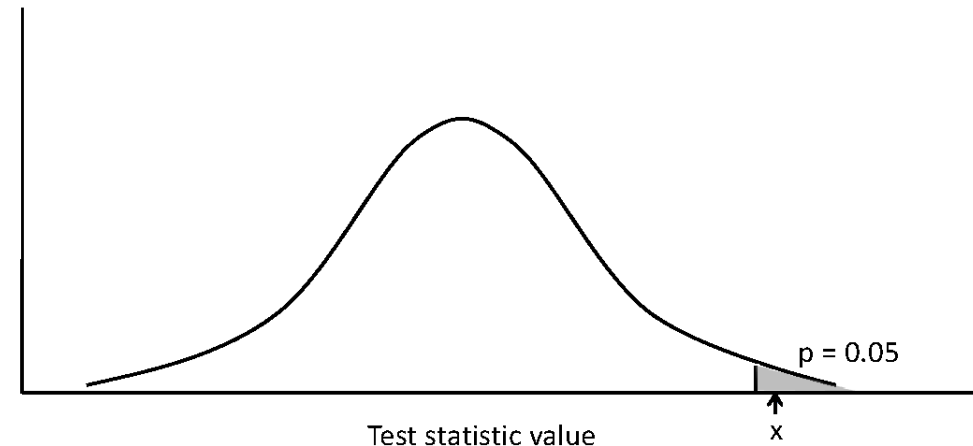
Significance Test Process

1. Compute the effectiveness measure for every query for both rankings.
2. Compute a *test statistic* based on a comparison of the effectiveness measures for each query. The test statistic depends on the significance test, and is simply a quantity calculated from the sample data that is used to decide whether or not the null hypothesis should be rejected.
3. The test statistic is used to compute a *P-value*, which is the probability that a test statistic value at least that extreme could be observed if the null hypothesis were true. Small P-values suggest that the null hypothesis may be false.
4. The null hypothesis (no difference) is rejected in favor of the alternate hypothesis (i.e., *B* is more effective than *A*) if the P-value is $\leq \alpha$, the *significance level*. Values for α are small, typically .05 and .1, to reduce the chance of a Type I error.

One-Sided or Two-sided Tests

- The right-hand table shows example experimental results, with *A* as the baseline model and *B* as the proposed new model.
- A one-sided or one-tailed test is used to establish that *B* is *better than* baseline *A*; and a two-sided or two-tailed test is used to establish that a *difference* between *B* and *A*.
 - where the “side” or “tail” *x* in the right-hand figure referred to is the tail of a probability distribution.
- The distribution curve in the figure is for the possible values of a test statistic assuming the null hypothesis
 - The shaded area *x* is the *region of rejection for a one-sided test*.
- The significance tests most commonly used are the *t-test*, the *Wilcoxon signed-rank test*, and the *sign test*.

Query	A	B	B-A
1	25	35	10
2	43	84	41
3	39	15	-24
4	75	75	0
5	43	68	25
6	15	85	70
7	20	80	60
8	52	50	-2
9	49	58	9
10	50	75	25



- Assumption is that the difference between the effectiveness values is a sample from a normal distribution
- Null hypothesis is that the mean of the distribution of differences is zero
- Test statistic (N is the sample size, e.g., the number of queries)

$$t = \frac{\overline{B-A}}{\sigma_{B-A}} \cdot \sqrt{N}$$

- for the example,

$$\overline{B-A} = 21.4, \sigma_{B-A} = 29.1, t = 2.33, \text{p-value} = .02$$

The mean of the differences

The standard deviation
of the differences

If the **p-value is very small**, then either the null hypothesis is false or something unlikely has occurred. In a formal *significance test*, the null hypothesis is rejected if the **p-value is less than** a predefined threshold value which is commonly set to **0.05**.

Example of t-Test in Excel

32

- In Excel, t-Test is described as follows:
 - If [Data > Data Analysis] does not exist, then do the following:
 - [File > Options > Add-Ins > Analysis ToolPak > Go > (click both “Analysis ToolPak” and “Solver Add-in” then OK)]
 - [Data > Data Analysis > t-Test: Paired Two Sample for Means]
 - Right dialogue box will appear.
- The figures illustrate an example using Excel for Mac. The p-value is **0.02249** for the **one-tailed test** and **0.04498** for the **two-tailed test**, both of which are less than **0.05**, indicating that the null hypothesis is rejected.
- Furthermore, the mean effectiveness of Variable 2 (Model B) is greater than that of Variable 1 (Model A); therefore, Model B performs statistically significantly better than Model A.

t-Test: Paired Two Sample for Means

Input

Variable 1 Range:

Variable 2 Range:

Hypothesized Mean Difference:

☐ Labels

Alpha:

Output options

☒ Output Range:

☐ New Worksheet Ply:

☐ New Workbook

OK Cancel

The screenshot shows the Excel Data Analysis toolpak dialog box. The 't-Test: Paired Two Sample for Means' option is selected. The background spreadsheet shows two columns of data: Model A and Model B. The data for Model A is [25, 43, 39, 75, 43, 15, 20, 52, 49, 50] and for Model B is [35, 84, 15, 75, 68, 85, 80, 50, 58, 75].

t-Test: Paired Two Sample for Means		
	Variable 1	Variable 2
Mean	41.1	62.5
Variance	311.8777778	534.055556
Observations	10	10
Pearson Correlation	0.000136126	
Hypothesized Mean Difference	0	
df	9	
t Stat	-2.32688129	
P(T<=t) one-tail	0.022488111	
t Critical one-tail	1.833112933	
P(T<=t) two-tail	0.044976221	
t Critical two-tail	2.262157163	

References

- Chapter 7 (sections 7.1, 7.2 and 7.3) in book - W. Bruce Croft, *Search Engines - Information retrieval in Practice*; Pearson, 2010.
- Chapter 1, "Probability", 1993, ISBN : 978-1-4612-6937-3, https://qut.primo.exlibrisgroup.com/permalink/61QUT_INST/1g7tbfa/alma991010463496504001
- Albishre K., Li Y., Xu Y. (2018) *Query-Based Automatic Training Set Selection for Microblog Retrieval*. In: Phung D., Tseng V., Webb G., Ho B., Ganji M., Rashidi L. (eds) *Advances in Knowledge Discovery and Data Mining. PAKDD 2018. Lecture Notes in Computer Science*, vol 10938. DOI https://doi.org/10.1007/978-3-319-93037-4_26
- Albishre, K., Li, Y. *et al.* Query-based unsupervised learning for improving social media search. *World Wide Web* **23**, 1791–1809 (2020). <https://doi.org/10.1007/s11280-019-00747-0>